

Тема 1. Повторение: компоненты, хуки, модульные стили

Организационное примечание. По расписанию лекция и первый семинар проходят в один день, но их порядок может отличаться: иногда лекция идёт первой, иногда — после первого семинара. Материал лекции написан так, чтобы быть самодостаточным независимо от этого порядка: он не полагается на то, что семинар уже прошёл.

Зачем это нужно

Ты уже писал React-приложения в прошлом семестре: собирал интерфейс из компонентов, работал с хуками, стилизовал через CSS-модули, доставал данные с сервера через React Query. С тех пор прошло время — и это нормально, что часть деталей стёрлась. Этот материал вернёт в рабочее состояние тот фундамент, на котором будет строиться весь остальной курс: без уверенного владения компонентами, хуками и стилями двигаться дальше — к серверному состоянию, backend и базам данных — не получится.

Если ты разберёшься с материалом этой темы, то:

- Освежишь в памяти, как устроен компонент, как в него попадают данные и как он реагирует на их изменение;
- Вспомнишь жизненный цикл компонента и то, как `useEffect` покрывает каждый его этап;
- Перестанешь путаться, когда использовать `useState`, а когда `useRef`, и почему стили из CSS-модулей не конфликтуют между собой.

Тебе для этой темы нужно помнить:

- Базовый синтаксис TypeScript: типы, интерфейсы, дженерики (пройдено в прошлом семестре);
- Как выглядит JSX-разметка и как запустить проект на Vite (`npm run dev`).

Быстрый повторительный блок

JSX

Определение

JSX — расширение синтаксиса JavaScript/TypeScript, позволяющее описывать разметку интерфейса прямо в коде, в форме, похожей на HTML. Каждый JSX-фрагмент — это обычное выражение языка: при сборке он превращается в вызов функции React, создающей описание элемента интерфейса.

Если сказать проще: JSX выглядит как HTML, вставленный в код, но это не строка и не HTML. Это выражение — его можно сохранить в переменную, вернуть из функции, вложить в другой JSX. А

внутри фигурных скобок `{}` пишется любое выражение TypeScript — именно так в разметку попадают данные:

```
const title = "Сделать дизайн";
const card = <div className="card">{title}</div>; // JSX-выражение, лежащее в переменной
```

Два отличия от HTML, о которые чаще всего спотыкаются: атрибут класса называется `className` (слово `class` в языке занято), а события пишутся в camelCase — `onClick`, а не `onclick`.

Компоненты

Определение

Компонент — это функция, которая принимает входные параметры (props) и возвращает JSX-разметку, описывающую фрагмент пользовательского интерфейса. React вызывает эту функцию каждый раз, когда компонент нужно отрисовать или перерисовать.

Если сказать проще: компонент — это кирпичик, из которого собирается страница. Кнопка — компонент. Карточка задачи — компонент. Колонка из карточек — тоже компонент, внутри которого лежат карточки. Вся страница — один большой компонент, собранный из маленьких. Смысл в том, что кирпичик описывается один раз, а используется сколько угодно раз и в каких угодно местах — как деталь конструктора.

```
type BoardCardProps = {
  title: string;
  isDone: boolean;
};

function BoardCard({ title, isDone }: BoardCardProps) {
  return <div className={isDone ? "card card--done" : "card"}>{title}</div>;
}
```

Компоненты вкладываются друг в друга, а данные передаются им сверху вниз через **props** — параметры, которые задаёт родитель:

```
function BoardColumn() {
  return (
    <div className="column">
      <BoardCard title="Сделать дизайн" isDone={true} />
      <BoardCard title="Написать API" isDone={false} />
    </div>
  );
}
```

Тип `BoardCardProps` здесь не для галочки: если забыть передать `isDone` или передать в `title` число вместо строки, компилятор поймает ошибку ещё до запуска приложения.

Условный рендеринг

Определение

Условный рендеринг — приём, при котором то, что вернёт компонент, зависит от условия: разные данные — разная разметка.

JSX — это обычные выражения TypeScript, поэтому внутри разметки работают привычные операторы. Два самых ходовых способа:

```
function BoardColumn({ cards }: { cards: Card[] }) {
  // Тернарный оператор — выбор между двумя вариантами:
  return (
    <div className="column">
      {cards.length > 0 ? (
        <span>Карточек: {cards.length}</span>
      ) : (
        <span>Колонка пуста</span>
      )}
    </div>
  );
}
```

```
function BoardCard({ title, isUrgent }: { title: string; isUrgent: boolean }) {
  // Оператор && — "показать или не показывать вовсе":
  return (
    <div className="card">
      {isUrgent && <span className="badge">Срочно</span>}
      {title}
    </div>
  );
}
```

Условие может определять и целиком возвращаемое значение компонента — например, ранний выход:

```
function CardDetails({ card }: { card: Card | null }) {
  if (!card) {
    return <p>Карточка не выбрана</p>;
  }
  return <article>{card.title}</article>;
}
```

Важно

Условный рендеринг применим не только к контенту, но и к **пропсам**: одному и тому же компоненту можно в зависимости от условия передавать разные значения. Разметка при этом не меняется — меняются данные, которые в неё уходят:

```
function BoardCard({ title, isDone, isUrgent }: BoardCardProps) {
  return (
    <div
      className={isDone ? styles.done : styles.card}
      title={isUrgent ? "Требуется внимания сегодня" : undefined}
      aria-disabled={isDone ? true : undefined}
    >
      {title}
    </div>
  );
}
```

Здесь по условию выбирается CSS-класс, а атрибуты `title` и `aria-disabled` при значении `undefined` вовсе не попадут в итоговый HTML — то есть условием можно не только менять значение пропса, но и решать, передавать ли его вообще.

Циклический рендеринг (рендеринг списков)

Определение

Циклический рендеринг — построение разметки из массива данных: каждому элементу массива соответствует свой фрагмент JSX. Стандартный инструмент для этого — метод массива `.map()`.

```
type Card = {
  id: string;
  title: string;
  isDone: boolean;
};

function BoardColumn({ cards }: { cards: Card[] }) {
  return (
    <div className="column">
      {cards.map((card) => (
        <BoardCard key={card.id} title={card.title} isDone={card.isDone} />
      ))}
    </div>
  );
}
```

Важно

Каждому элементу списка нужен уникальный пропс `key` — по нему React при обновлении понимает, какой элемент добавился, какой удался, а какой просто переместился, и перерисовывает только то, что реально изменилось. Использовать индекс массива как `key` — плохая привычка: при удалении или перестановке элементов индексы "съезжают", и React может перепутать элементы между собой (например, состояние поля ввода одной карточки "переедет" в другую). Правильный `key` — стабильный идентификатор из самих данных, как `card.id` выше.

Условный и циклический рендеринг свободно комбинируются — например, отрисовать список и подсветить только срочные:

```
{
  cards.map((card) =>
    card.isUrgent ? (
      <BoardCard
        key={card.id}
        title={`🔥 ${card.title}`}
        isDone={card.isDone}
      />
    ) : (
      <BoardCard key={card.id} title={card.title} isDone={card.isDone} />
    ),
  );
}
```

Жизненный цикл компонента

Определение

Жизненный цикл компонента — последовательность этапов, через которые проходит компонент: **монтирование** (первое появление на странице), **обновление** (перерисовка из-за изменения props или состояния) и **размонтирование** (удаление со страницы).

Каждому этапу соответствует момент, когда компоненту может понадобиться что-то сделать: при появлении — запустить таймер или подписаться на событие, при обновлении — среагировать на изменившиеся данные, при удалении — прибраться за собой (остановить таймер, отписаться), иначе приложение начнёт "течь".

В старом классовом React для этого были отдельные методы (`componentDidMount` , `componentDidUpdate` , `componentWillUnmount`). В современном функциональном React все три этапа покрывает один хук — `useEffect` , а какой именно этап он обслуживает, определяется его форматом:

Этап жизненного цикла	Классовый метод (для справки)	Формат <code>useEffect</code>
Монтирование	<code>componentDidMount</code>	<code>useEffect(() => { ... }, [])</code> — пустой массив зависимостей: выполнится один раз при появлении
Обновление	<code>componentDidUpdate</code>	<code>useEffect(() => { ... }, [value])</code> — выполнится при монтировании и затем при каждом изменении <code>value</code>
Размонтирование	<code>componentWillUnmount</code>	<code>useEffect(() => { return () => { ... }; }, [])</code> — функция, возвращённая из эффекта, выполнится при удалении компонента

Все три формата на одном примере:

```
function BoardTimer({ boardId }: { boardId: string }) {
  // Монтирование: выполнится один раз при появлении компонента
```

```

useEffect(() => {
  console.log("Компонент появился на странице");
}, []);

// Обновление: выполнится при монтировании и при каждой смене boardId
useEffect(() => {
  console.log(`Открыта доска ${boardId}`);
}, [boardId]);

// Размонтирование: возвращённая функция выполнится при удалении компонента
useEffect(() => {
  const timerId = setInterval(() => console.log("тик"), 1000);
  return () => clearInterval(timerId); // очистка — иначе таймер переживёт компонент
}, []);

return <div>Доска {boardId}</div>;
}

```

Важно

Функция очистки (возвращённая из эффекта) выполняется не только при размонтировании, но и **перед каждым повторным запуском эффекта** при обновлении зависимостей. Это защищает от накопления подписок и таймеров: старый убирается, прежде чем создаётся новый.

Хуки

Определение

Хук — специальная функция React (её имя всегда начинается с `use`), которая подключает к компоненту дополнительную возможность: хранить состояние, выполнять побочные эффекты, запоминать значение между рендерами.

Компонент-функция сам по себе "забывает" всё при каждом вызове — хуки как раз дают ему память и связь с внешним миром. Три основных:

useState — **хранение состояния**. Значение, которое переживает перерисовки, и функция для его изменения. Изменение состояния — единственный способ заставить React перерисовать компонент с новыми данными:

```

function CardCounter() {
  const [count, setCount] = useState<number>(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Карточек добавлено: {count}
    </button>
  );
}

```

useEffect — **побочные эффекты**. Код, который должен выполняться *после* отрисовки: подписка на события, установка заголовка вкладки, таймеры. Как он соотносится с этапами жизненного цикла —

разобрано в разделе выше; здесь напомним типичный вид:

```
function BoardPage({ boardName }: { boardName: string }) {
  useEffect(() => {
    document.title = `Доска: ${boardName}`;
  }, [boardName]); // выполнится при монтировании и при каждом изменении boardName

  return <h1>{boardName}</h1>;
}
```

useRef — значение без перерисовки. Контейнер, в котором можно хранить что угодно между рендерами, но изменение которого *не* вызывает перерисовку. Чаще всего используется для доступа к DOM-элементу напрямую:

```
function NewCardForm() {
  const inputRef = useRef<HTMLInputElement>(null);

  const focusInput = () => {
    inputRef.current?.focus(); // прямое обращение к DOM-элементу
  };

  return (
    <>
      <input ref={inputRef} placeholder="Название карточки" />
      <button onClick={focusInput}>Начать ввод</button>
    </>
  );
}
```

Важно

Главное различие: изменение `useState` перерисовывает компонент, изменение `useRef.current` — нет. Если значение должно отображаться на экране — это `useState`. Если оно нужно только "за кулисами" (ссылка на DOM, id таймера) — это `useRef`.

Модульные стили (CSS Modules)

Определение

CSS-модуль — файл стилей с именем `*.module.css`, классы из которого при сборке автоматически получают уникальные имена. Благодаря этому стили одного компонента физически не могут повлиять на другой.

Проблема обычного CSS: все классы живут в одном глобальном пространстве имён. Если в двух разных файлах объявлен `.card`, они перезапишут друг друга — и выяснить, чей стиль "победил", бывает мучительно. CSS-модули решают это на уровне сборщика.

`BoardCard.module.css` :

```
.card {
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 12px;
}

.done {
  opacity: 0.5;
  text-decoration: line-through;
}
```

BoardCard.tsx :

```
import styles from './BoardCard.module.css';

function BoardCard({ title, isDone }: BoardCardProps) {
  return (
    <div className={isDone ? `${styles.card} ${styles.done}` : styles.card}>
      {title}
    </div>
  );
}
```

Во что это превращается после сборки: Vite сгенерирует для каждого класса уникальное имя, а объект `styles` станет простым словарём "имя в коде → имя в собранном CSS":

```
<!-- в итоговом HTML -->
<div class="_card_1a2b3_1 _done_1a2b3_7">Сделать дизайн</div>
```

```
/* в итоговом CSS */
._card_1a2b3_1 {
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 12px;
}
```

Поэтому `.card` из `BoardCard.module.css` и `.card` из любого другого модуля — это два разных класса в итоговой сборке, и конфликт между ними невозможен в принципе.

Клиентское и серверное состояние

В повторительном блоке слово "состояние" встречалось в одном смысле — значение внутри `useState`, живущее в конкретном компоненте. Прежде чем идти дальше, это понятие нужно

расширить: сейчас речь пойдёт о классификации **данных всего приложения**, а не о механизме их хранения в компоненте.

Важно

Не путай две разные вещи. **Состояние компонента** (`useState`) — это *механизм*: способ, которым компонент хранит значение между перерисовками. **Клиентское и серверное состояние** — это *классификация данных* по тому, кто является их источником истины. Данные любого из двух видов физически могут лежать в `useState` — вопрос "клиентское или серверное" не про то, *где* данные хранятся, а про то, *кому они принадлежат*.

Определение

Серверное состояние (server state) — данные, источником истины для которых является сервер (его база данных). Приложение в браузере не владеет ими, а лишь запрашивает и временно отображает их копию — которая с момента получения может устареть, потому что данные на сервере способны измениться без ведома этого браузера: другим пользователем, другим устройством, фоновым процессом.

Клиентское состояние (client state) — данные, источником истины для которых является само приложение в браузере. Они рождаются и живут только в текущей вкладке, сервер о них не знает и знать не обязан. Такие данные не могут "устареть" — они в любой момент ровно такие, какими их сделал пользователь этой вкладки.

Разница видна на примерах из канбан-доски:

Данные	Вид состояния	Почему
Список досок, колонок, карточек	Серверное	Хранится в БД; другой пользователь может добавить карточку — и наша копия устареет
Имя и роль текущего пользователя	Серверное	Источник истины — запись в БД на сервере
Открыто ли модальное окно карточки	Клиентское	Существует только в этой вкладке; серверу безразлично
Выбранный фильтр "показать только мои задачи"	Клиентское	Настройка текущего сеанса работы, сервер о ней не знает
Текст новой карточки, который печатается прямо сейчас, но ещё не сохранён	Клиентское	Пока не нажата кнопка "Создать" — данных на сервере нет, истина только в браузере

Обрати внимание на последнюю строку: тот же текст карточки **после** сохранения станет серверным состоянием — источник истины переедет в БД. То есть одни и те же по смыслу данные могут менять вид состояния в зависимости от того, на каком этапе своей жизни они находятся.

Ключевой вопрос для классификации всегда один: **если закрыть вкладку и открыть приложение заново — эти данные должны восстановиться с сервера?** Если да — состояние серверное. Если они по праву исчезают вместе с вкладкой — клиентское.

React Query — инструмент для серверного состояния

Серверным состоянием ты уже управлял в прошлом семестре — через React Query (TanStack Query). Напомним самое основное: запросы и мутации.

Определение

React Query — библиотека для управления серверным состоянием: она выполняет запросы к серверу, кэширует их результаты по ключам, отслеживает устаревание данных и умеет автоматически перезапрашивать их, когда это нужно.

Если сказать проще: React Query — это посредник между твоими компонентами и сервером, который избавляет от рутины. Без него для каждого запроса пришлось бы вручную заводить `useState` для данных, ещё один — для флага загрузки, ещё один — для ошибки, и `useEffect`, чтобы всё это запустить. React Query сворачивает всю эту конструкцию в один хук и добавляет то, что вручную писать никто не станет: кэш, дедупликацию одинаковых запросов, фоновое обновление.

Запросы: `useQuery`

Определение

Запрос (query) — операция *чтения* серверного состояния. Описывается ключом (`queryKey`), по которому результат кладётся в кэш, и функцией (`queryFn`), которая непосредственно ходит за данными.

Неформально: запрос — это «дай посмотреть». Он ничего не меняет на сервере, сколько бы раз ни выполнялся, — поэтому React Query может свободно повторять его за кадром, не спрашивая тебя.

```
import { useQuery } from "@tanstack/react-query";

type Card = {
  id: string;
  title: string;
  isDone: boolean;
};

async function fetchCards(): Promise<Card[]> {
  const response = await fetch("/api/cards");
  if (!response.ok) throw new Error("Не удалось загрузить карточки");
  return response.json();
}

function BoardColumn() {
  const {
    data: cards,
    isLoading,
    isError,
  } = useQuery({
    queryKey: ["cards"],
    queryFn: fetchCards,
  });
}
```

```

if (isLoading) return <p>Загрузка...</p>;
if (isError) return <p>Что-то пошло не так</p>;

return (
  <div className="column">
    {cards?.map((card) => (
      <BoardCard key={card.id} title={card.title} isDone={card.isDone} />
    ))}
  </div>
);
}

```

В чём смысл этого кода: компонент не «делает запрос», а **декларирует потребность в данных** с ключом `["cards"]`. Всё остальное — забота React Query. Если другой компонент на той же странице тоже попросит `["cards"]`, второго запроса к серверу не будет — оба получают данные из одного кэша. Состояния загрузки и ошибки (`isLoading`, `isError`) приходят из хука готовыми — никаких ручных `useState` под них.

Важно

`queryKey` — это не просто имя, а **адрес данных в кэше**. Один и тот же ключ по всему приложению означает одни и те же данные. Для данных, зависящих от параметра, параметр включается в ключ: `["cards", boardId]` — карточки каждой доски кэшируются отдельно, и смена `boardId` автоматически приведёт к запросу нужных данных.

Мутации: `useMutation`

Определение

Мутация (mutation) — операция *изменения* серверного состояния: создание, обновление или удаление данных. В отличие от запроса, выполняется только по явной команде (`mutate`) и никогда не повторяется автоматически.

Неформально: мутация — это «измени». Повторить её без спроса нельзя: два автоматических повтора «создай карточку» — это две карточки. Поэтому мутация запускается только тогда, когда её явно вызвали — например, по клику на кнопку.

```

import { useMutation, useQueryClient } from "@tanstack/react-query";

async function createCard(title: string): Promise<Card> {
  const response = await fetch("/api/cards", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title }),
  });
  if (!response.ok) throw new Error("Не удалось создать карточку");
  return response.json();
}

function NewCardButton() {
  const queryClient = useQueryClient();

```

```

const { mutate, isPending } = useMutation({
  mutationFn: createCard,
  onSuccess: () => {
    // Сообщаем кэшу: данные под ключом ["cards"] устарели — перезапроси
    queryClient.invalidateQueries({ queryKey: ["cards"] });
  },
});

return (
  <button onClick={() => mutate("Новая карточка")} disabled={isPending}>
    {isPending ? "Создаём..." : "Добавить карточку"}
  </button>
);
}

```

В чём смысл этого кода: сама мутация (`mutationFn`) только отправляет изменение на сервер — но после успеха список карточек в кэше уже не соответствует реальности, ведь на сервере появилась новая. Строка с `invalidateQueries` — самая важная в примере: она **помечает данные под ключом `["cards"]` устаревшими**, и React Query сам их перезапрашивает. Компонент `BoardColumn` из примера выше при этом обновится автоматически — хотя он ничего не знает о существовании кнопки.

Важно

Связка «мутация → инвалидация → автоматический перезапрос» — главный рабочий цикл React Query. Частая ошибка — после мутации вручную обновлять какой-нибудь локальный список в `useState` : так появляется вторая копия серверных данных, которая быстро расходится с кэшем.

Итог по разделу: пара «запрос — чтение, мутация — изменение» полностью закрывает работу с серверным состоянием. Но для клиентского состояния — того, о котором сервер не знает, — React Query не предназначен. Об инструменте для него — дальше.

Zustand — инструмент для клиентского состояния

Что такое стор

Определение

Стор (store, хранилище) — объект, который живёт вне дерева компонентов и содержит состояние вместе с функциями для его изменения. Компоненты подписываются на стор и перерисовываются, когда меняются данные, которые они из него читают.

Неформально: стор — это общая полка. `useState` — это карман конкретного компонента: чтобы содержимым кармана поделиться, его нужно передавать из рук в руки через props. Стор же стоит в стороне от всех — любой компонент, где бы он ни находился в дереве, может подойти к полке, взять с неё значение или положить новое. Кто взял — тот узнает, когда содержимое изменится.

Что такое Zustand

Определение

Zustand — минималистичная библиотека для создания сторов клиентского состояния в React-приложениях. Стор создаётся одной функцией `create`, используется как обычный хук и не требует оборачивания приложения в `Provider`.

Зачем он нужен

Вспомни, чем заканчивался раздел про клиентское состояние: открытая модалка, выбранный фильтр, свёрнутая колонка. Пока такое значение нужно одному компоненту — `useState` на своём месте, и ничего лучше не надо.

Проблема начинается, когда одно и то же клиентское состояние нужно **нескольким компонентам в разных концах дерева**. Кнопка «фильтр: только мои задачи» живёт в шапке страницы, а применяется фильтр в колонках доски — между ними несколько уровней вложенности. Средствами самого React есть два пути, и оба с изъяном:

- **Поднять состояние вверх и прокинуть через props.** Работает, но пропс поедет через цепочку компонентов, которым он сам не нужен — они лишь передают его дальше. Это называют `prop drilling`: каждый промежуточный компонент обрастает чужими пропсами и становится хрупким — переставил один уровень дерева, переписывай всю цепочку.
- **Положить в Context.** Прокидывать ничего не нужно, но у контекста есть цена: при изменении его значения перерисовываются **все** компоненты, подписанные через `useContext`, — даже те, кому изменившееся поле безразлично. Один контекст с фильтром, темой и открытой модалкой означает, что переключение темы перерисует и колонки, читавшие оттуда только фильтр.

Zustand закрывает оба изъяна разом: состояние лежит вне дерева (никакого `prop drilling`), а компонент подписывается не на весь стор, а на конкретное значение из него — и перерисовывается только когда меняется именно оно.

Пример использования

Стор фильтра для канбан-доски — файл `store/boardFilters.ts`:

```
import { create } from "zustand";

type BoardFiltersState = {
  showOnlyMyCards: boolean;
  toggleOnlyMyCards: () => void;
};

export const useBoardFiltersStore = create<BoardFiltersState>((set) => ({
  showOnlyMyCards: false,
  toggleOnlyMyCards: () =>
    set((state) => ({ showOnlyMyCards: !state.showOnlyMyCards })),
}));
```

Использование в двух компонентах, находящихся в разных концах дерева:

```
// Шапка страницы — переключает фильтр
function BoardHeader() {
  const toggleOnlyMyCards = useBoardFiltersStore(
    (state) => state.toggleOnlyMyCards,
  );

  return <button onClick={toggleOnlyMyCards}>Только мои задачи</button>;
}

// Колонка доски — читает фильтр
function BoardColumn({ cards, currentUserId }: BoardColumnProps) {
  const showOnlyMyCards = useBoardFiltersStore(
    (state) => state.showOnlyMyCards,
  );

  const visibleCards = showOnlyMyCards
    ? cards.filter((card) => card.authorId === currentUserId)
    : cards;

  return (
    <div className="column">
      {visibleCards.map((card) => (
        <BoardCard key={card.id} title={card.title} isDone={card.isDone} />
      ))}
    </div>
  );
}
```

Как это работает, по шагам:

1. **create** **вызывается один раз при загрузке модуля** — и создаёт стор: объект с состоянием (`showOnlyMyCards: false`) и функциями его изменения (`toggleOnlyMyCards`). Стор существует в единственном экземпляре независимо от того, сколько компонентов им пользуются и смонтированы ли они вообще.
2. **create** **возвращает хук** (`useBoardFiltersStore`). Компонент вызывает его, передавая **селектор** — функцию, которая говорит, *какая именно часть стора этому компоненту нужна*: `(state) => state.showOnlyMyCards`.
3. **Вызов хука с селектором — это подписка**. С этого момента Zustand следит: изменилось значение, которое возвращает селектор, — компонент перерисовывается; изменилось что-то другое в сторе — компонент не трогается.
4. **Изменение состояния идёт только через set**. Функция `toggleOnlyMyCards` внутри стора вызывает `set`, тот вычисляет новое состояние и уведомляет подписчиков, чьи селекторы вернули новое значение. Клик по кнопке в `BoardHeader` перерисует `BoardColumn` — хотя эти компоненты не связаны ни props, ни общим родителем ниже корня.

Важно

Селектор — не украшение, а механизм точной подписки. Вызов `useBoardFiltersStore()` без селектора подпишет компонент на весь стор — и он будет перерисовываться при любом изменении любого поля, ровно как с `useContext`. Правило простое: один селектор — одно конкретное значение, которое компоненту действительно нужно.

Где проходит граница между React Query и Zustand

Теперь оба инструмента на столе, и главный практический вопрос темы звучит так: **какие данные куда класть?** Ответ уже был дан в разделе про виды состояния — инструмент выбирается по источнику истины, а не по удобству момента:

	React Query	Zustand
Вид состояния	Серверное	Клиентское
Источник истины	Сервер (БД)	Текущая вкладка браузера
Данные могут устареть	Да — поэтому есть кэш, инвалидация, перезапрос	Нет — они всегда ровно такие, какими их сделал пользователь
Как данные меняются	Мутация → инвалидация → перезапрос	Вызов экшена стора → <code>set</code>
На проекте канбан-доски	Доски, колонки, карточки, текущий пользователь	Открытая модалка, фильтры, свёрнутые колонки, тема оформления

Правило на каждый день — тот же вопрос, что и при классификации состояния:

Совет

Прежде чем заводить поле в сторе Zustand, спроси себя: «можно ли это значение восстановить, просто запросив его у сервера?» Если да — этому значению место в React Query, и в Zustand его класть не нужно. Если нет — это клиентское состояние, и Zustand для него подходит.

А вот чего делать нельзя ни при каких обстоятельствах:

Предостережение

Не копируй данные из React Query в Zustand «для удобства». Как только список карточек оказывается и в кэше React Query, и в сторе Zustand — у приложения два источника истины. После первой же мутации кэш перезапросится и обновится, а копия в сторе останется старой: на одном экране будут одни данные, на другом — другие, и такой баг очень тяжело искать. Данные с сервера читаются из `useQuery` напрямую, всегда.

Если из данных React Query нужно что-то *вычислить* с участием клиентского состояния (например, отфильтровать карточки по значению фильтра из Zustand) — это делается прямо в компоненте: взять `data` из `useQuery`, взять значение фильтра из стора, применить `filter`. Результат вычисления не хранится нигде — он каждый раз выводится из двух своих источников.

Необходимо знать для дальнейшего: хардкод, мок и мок-сервер

Два слова, которые будут постоянно встречаться на семинарах. Разберём оба.

Хардкод

Определение

Хардкод (hardcode, «жёстко зашитое») — данные, записанные прямо в коде программы, вместо того чтобы приходить извне: из файла, с сервера, от пользователя. «Захардкодить» — вписать значение в код намертво.

Если сказать проще: массив карточек, объявленный прямо внутри компонента, — это хардкод. Чтобы изменить такие данные, нужно править код и пересобирать приложение; при перезагрузке страницы всё добавленное пользователем исчезает — данные каждый раз рождаются заново из кода.

Хардкод — это не ругательство, а нормальный первый шаг: «сначала заставим интерфейс работать на простых данных, потом подключим настоящие». Ровно так построены семинары курса: доска начинается с захардкоженного массива, а затем он заменяется данными с сервера.

Мок и мок-сервер

На семинарах этой и следующих тем, пока у канбан-доски не появится настоящий сервер, его роль будет играть **мок-сервер**.

Определение

Мок (mock, англ. «имитация») — упрощённая замена настоящей части системы, которая снаружи ведёт себя как настоящая: принимает те же запросы и отвечает в том же формате, но внутри устроена примитивно. **Мок-сервер** — мок целого сервера: он отвечает на те же HTTP-запросы, что и будущий настоящий backend, только вместо базы данных и бизнес-логики у него, как правило, обычный файл с данными.

Если сказать проще: мок — это картонная декорация сервера. С места зрителя (то есть клиента) она неотличима от настоящего здания: те же адреса (`/api/cards`), те же ответы в JSON. Но за фасадом нет ни базы данных, ни проверки прав, ни логики — только заранее заготовленные данные.

Зачем нужна такая декорация:

- **Клиент можно разрабатывать до того, как готов сервер.** В командах фронтенд и бэкенд часто пишутся параллельно: договорились о формате данных — и фронтендер работает с моком, не дожидаясь бэкендера. Наш курс устроен так же: настоящий сервер мы напишем в следующих темах, а клиенту данные нужны уже сейчас.
- **Клиентский код не отличает мок от настоящего сервера.** `fetch("/api/cards")` и React Query работают одинаково с тем и с другим. Когда в курсе появится настоящий backend, мы просто выключим мок — и в коде клиента не изменится ни строчки. Это главный признак правильно устроенного мока.

В проекте курса мок-сервер — это **json-server**: маленькая библиотека, которая превращает JSON-файл в REST API одной командой. Данные лежат в файле `db.json`:

```
{
  "cards": [
    { "id": "1", "title": "Спроектировать структуру доски", "isDone": true },
    { "id": "2", "title": "Подключить доску к серверу", "isDone": false }
  ]
}
```

После запуска (в проекте курса — команда `npm run server`) json-server сам делает из этого файла работающий API:

Запрос	Что делает
GET /cards	вернёт массив карточек из файла
POST /cards	добавит карточку в файл и вернёт её с новым <code>id</code>
PATCH /cards/1	изменит карточку с <code>id: 1</code>
DELETE /cards/1	удалит карточку с <code>id: 1</code>

Причём изменения он честно записывает обратно в `db.json` — поэтому созданная карточка переживает перезагрузку страницы, как и положено серверным данным.

Важно

Мок — это именно *отдельный сервер-имитация*, живущий за пределами клиентского кода. Захардкоженный массив карточек внутри компонента — не мок: клиент, привязанный к такому массиву, придётся переписывать при переходе на сервер. Клиент, который ходит к мок-серверу по HTTP, при замене мока на настоящий сервер не меняется вообще.

Частые ошибки новичков

- **Дублирование серверных данных в Zustand** — см. предупреждение выше: два источника истины, рассинхронизация, трудноуловимые баги.
- **Подписка на весь стор** (`useStore()` без селектора) — компонент перерисовывается при любом изменении любого поля, и преимущество перед Context теряется.
- **Один гигантский стор на всё приложение** — фильтры доски, тема оформления и состояние модалок сваливаются в одну кучу. Правильнее несколько маленьких сторов по смыслу: `useBoardFiltersStore`, `useUiStore` — каждый про своё.
- **Изменение состояния мимо `set`** — прямое присваивание в объект состояния не уведомит подписчиков, и интерфейс просто не обновится.
- **Клиентское состояние «на всякий случай» в сторе, когда хватает `useState`** — если значение нужно одному-единственному компоненту (текст в поле ввода, раскрыт ли локальный список),

обычный `useState` проще и правильнее. Стор — для того, что нужно нескольким компонентам в разных местах дерева.

Заключение

Собери тему в одну картину:

- Компоненты, условный и циклический рендеринг, жизненный цикл, хуки и CSS-модули — фундамент из прошлого семестра, на котором строится всё дальнейшее;
- Данные приложения делятся на **серверные** (источник истины — сервер, копия в браузере может устареть) и **клиентские** (источник истины — сама вкладка);
- Для серверного состояния — **React Query**: запрос читает, мутация изменяет, инвалидация запускает перезапрос;
- Для клиентского состояния, нужного нескольким компонентам, — **Zustand**: стор вне дерева, точная подписка через селекторы, изменение через `set` ;
- Граница между ними — вопрос «восстановится ли это с сервера?», и пересекать её копированием данных нельзя.

Эта схема разделения будет каркасом всего курса: в следующих темах мы построим сервер, который и станет тем самым «источником истины» для серверного состояния, — а клиент канбан-доски уже готов принимать его данные правильно.

Дополнительные материалы (необязательно, для желающих углубиться)

- Официальная документация Zustand: <https://zustand.docs.pmnd.rs/>
- Официальная документация TanStack Query: <https://tanstack.com/query/latest/docs/framework/react/overview>
- TkDodo (мейнтейнер React Query), серия статей «Practical React Query»: <https://tkdodo.eu/blog/practical-react-query> — в том числе статья «Zustand and React Query» о совместном использовании
- Kent C. Dodds, «Application State Management with React»: <https://kentcdodds.com/blog/application-state-management-with-react> — какие виды состояния бывают и почему не всё состояние глобальное
- Документация React, раздел «Managing State»: <https://react.dev/learn/managing-state> — систематизация подходов к состоянию средствами самого React